JavaScript is disabled on your browser.

**Java SE 27 & JDK 27**

## Interface Formattable

`public interface` **`Formattable`**

The `Formattable` interface must be implemented by any class that needs to perform custom formatting using the `'s'` conversion specifier of `Formatter`. This interface allows basic control for formatting arbitrary objects. For example, the following class prints out different representations of a stock's name depending on the flags and length constraints:

```
import java.nio.CharBuffer;
import java.util.Formatter;
import java.util.Formattable;
import java.util.Locale;
import static java.util.FormattableFlags.*;

...

public class StockName implements Formattable {
    private String symbol, companyName, frenchCompanyName;
    public StockName(String symbol, String companyName,
                     String frenchCompanyName) {
        ...
    }

    ...

    public void formatTo(Formatter fmt, int f, int width, int precision) {
        StringBuilder sb = new StringBuilder();

        // decide form of name
        String name = companyName;
        if (fmt.locale().equals(Locale.FRANCE))
            name = frenchCompanyName;
        boolean alternate = (f & ALTERNATE) == ALTERNATE;
        boolean usesymbol = alternate || (precision != -1 && precision < 10);
        String out = (usesymbol ? symbol : name);

        // apply precision
        if (precision == -1 || out.length() < precision) {
            // write it all
            sb.append(out);
        } else {
```

```
                sb.append(out.substring(0, precision - 1)).append('*');
            }

            // apply width and justification
            int len = sb.length();
            if (len < width)
                for (int i = 0; i < width - len; i++)
                    if ((f & LEFT_JUSTIFY) == LEFT_JUSTIFY)
                        sb.append(' ');
                    else
                        sb.insert(0, ' ');

            fmt.format(sb.toString());
        }

        public String toString() {
            return String.format("%s - %s", symbol, companyName);
        }
    }
```

When used in conjunction with the `Formatter`, the above class produces the following output for various format strings.

```
    Formatter fmt = new Formatter();
    StockName sn = new StockName("HUGE", "Huge Fruit, Inc.",
                                 "Fruit Titanesque, Inc.");
    fmt.format("%s", sn);                    //   -> "Huge Fruit, Inc."
    fmt.format("%s", sn.toString());         //   -> "HUGE - Huge Fruit, Inc."
    fmt.format("%#s", sn);                   //   -> "HUGE"
    fmt.format("%-10.8s", sn);               //   -> "HUGE      "
    fmt.format("%.12s", sn);                 //   -> "Huge Fruit,*"
    fmt.format(Locale.FRANCE, "%25s", sn);   //   -> "   Fruit Titanesque, Inc."
```

Formattables are not necessarily safe for multithreaded access. Thread safety is optional and may be enforced by classes that extend and implement this interface.

Unless otherwise specified, passing a `null` argument to any method in this interface will cause a `NullPointerException` to be thrown.

**Since:**

1.5

## Method Summary

| All Methods | Instance Methods | |
|---|---|---|

| Modifier and Type | Method | Description |
|---|---|---|
| void | **formatTo**(Formatter formatter, int flags, int width, int precision) | Formats the object using the provided formatter. |

## Method Details

### formatTo

```
void formatTo(Formatter formatter,
              int flags,
              int width,
              int precision)
```

Formats the object using the provided formatter.

**Parameters:**

formatter - The formatter. Implementing classes may call formatter.out() or formatter.locale() to obtain the Appendable or Locale used by this formatter respectively.

flags - The flags modify the output format. The value is interpreted as a bitmask. Any combination of the following flags may be set: FormattableFlags.LEFT_JUSTIFY, FormattableFlags.UPPERCASE, and FormattableFlags.ALTERNATE. If no flags are set, the default formatting of the implementing class will apply.

width - The minimum number of characters to be written to the output. If the length of the converted value is less than the width then the output will be padded by ' ' until the total number of characters equals width. The padding is at the beginning by default. If the FormattableFlags.LEFT_JUSTIFY flag is set then the padding will be at the end. If width is -1 then there is no minimum.

precision - The maximum number of characters to be written to the output. The precision is applied before the width, thus the output will be truncated to precision characters even if the width is greater than the precision. If precision is -1 then there is no explicit limit on the number of characters.

**Throws:**

`IllegalFormatException` - If any of the parameters are invalid. For specification of all possible formatting errors, see the Details section of the formatter class specification.

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.